

FinInsights

A KPI intelligence-to-action engine for a bank, and the bank that feeds it.

Start here — running the whole thing from scratch

Written for someone who has never seen this repository. Follow it top to bottom. Every command is copy-paste. Nothing needs to be installed on your machine except Docker and Ollama.

What you need first

Thing	Why	Where
Docker Desktop	Every service runs in a container. Nothing is installed on your machine.	docker.com/products/docker-desktop
Ollama	Runs the small language model locally, so no data ever leaves the machine.	ollama.com/download
~10 GB free disk	Docker images and the warehouse.	

Start Docker Desktop and wait until it says **Engine running**. Nothing below works until it does.

Step 1 — Install Ollama and pull the narrator model

The language model never produces a number. It reads the question, plans the investigation, and writes the final sentences from figures the engine already computed and stored. It is small on purpose: it runs on a laptop, costs nothing, and the bank's data never leaves the machine.

Install Ollama. Download the installer for your OS from ollama.com/download and run it. On Windows it installs as a normal application and starts by itself. On macOS, drag it to Applications and open it once.

Check it is running:

```
ollama --version
curl http://localhost:11434/api/tags      # should return JSON, not a connection error
```

Pull the model (about 1.9 GB, one time):

```
ollama pull qwen2.5:3b-instruct
```

Build the narrator. This is required, not optional. Ollama's default context is 2,048 tokens and it truncates a longer prompt **silently** rather than refusing. The narrator sends roughly 4,500 tokens of evidence before it writes a word, so on the default it would narrate from a claim set with the middle cut out, and you would never see an error.

```
# From the repository root
printf 'FROM qwen2.5:3b-instruct
PARAMETER num_ctx 8192
' > Modelfile
ollama create fininsights-narrator -f Modelfile
```

Confirm it exists:

```
ollama list          # fininsights-narrator should be in the list
```

Step 2 — Get the code and configure it

```
git clone <repository-url>
cd FinInsights
cp .env.example .env
```

You do **not** need to edit `.env` to run the demo. The defaults already point the stack at the Ollama running on your own machine:

```
VLLM_URL=http://host.docker.internal:11434/v1  # this is Ollama's port, 11434
INTELLIGENCE_LLM=1                             # 1 = narrator on, 0 = numbers only
```

The variable is still called `VLLM_URL` because an earlier version used a different server. The address is Ollama's `host.docker.internal` is how a container reaches a program running on your machine — that is why Ollama is installed normally rather than in Docker.

Step 3 — Start everything

```
docker compose up -d
```

The first run downloads images and builds the services. Expect **5 to 15 minutes** on a normal connection. After that it starts in under a minute.

Watch it come up:

```
docker compose ps
```

Wait until every service reads `running`. Twelve containers start: Kafka and Zookeeper, ClickHouse, the migration job, ingestion, the pipeline, the intelligence service, the analytics API, the dashboard, and the two NexaBank services.

Two things worth knowing:

- **Do not add `--profile gpu`**. That starts an old vLLM server this setup no longer uses, and it will fail or eat your GPU memory for nothing.

- **The `migrate` container exits on purpose.** It applies the warehouse schema and stops. Exit code 0 is success, not a crash.

Quick check that the core is alive:

```
curl localhost:8001/health      # analytics API
curl localhost:8000/health     # ingestion – shows kafka or clickhouse_fallback
```

Step 4 — Load the data

The bank starts empty. This seeds several months of realistic banking activity and plants the demo anomalies **in the source data**, so the engine has to discover them rather than being told:

```
docker compose --profile tools run --rm tools python scripts/seed_data.py --scenario all
```

This takes a few minutes and prints what it planted. It also writes `fixtures/planted_truth.json` — the ground truth, so you can check later whether the engine found what was actually put there.

Now build the warehouse layers. This reads the bank's database, cleans the data, and computes the KPIs into the Gold layer:

```
curl -X POST localhost:8003/refresh
```

Then score the findings. Each range is scored separately, because a 7-day finding and a 90-day finding are different claims about different windows:

```
curl -X POST "localhost:8001/intelligence/rescore?tenants=nexabank&days=7"
curl -X POST "localhost:8001/intelligence/rescore?tenants=nexabank&days=30"
curl -X POST "localhost:8001/intelligence/rescore?tenants=nexabank&days=90"
```

Each returns `{"ok": true, ...}`. Run all three, or the range selector on the dashboard will have nothing to show for the ranges you skipped.

Confirm there is data before opening the browser:

```
curl "localhost:8001/intelligence/insights?tenants=nexabank&days=30"
```

You should get a list of findings, one per KPI. An empty list means step 4 has not finished.

Step 5 — Open it

Open this	What you are looking at
http://localhost:3001/dashboard	The five KPIs, their trends, and how each sits against its expected range
http://localhost:3001/intelligence	The evidence page: what moved, why, the recommended action, and the full audit trail
http://localhost:3002	NexaBank itself — the retail bank generating the data
http://localhost:3002/admin/simulate	Plant a new anomaly and watch the engine find it

Step 6 — The five-minute demo

1. Open <http://localhost:3001/intelligence>. Pick a persona at the top — CFO, Operations Manager, Risk Officer, Analyst. Every persona sees every metric; what changes is the answer.
2. In **Ask the analyst**, type: *Why did KYC completion rate fall, and where is it concentrated?* The answer names the segment, shows each driver's contribution, and proposes an action with a named owner. Open **How this answer was derived** to see which step was SQL, statistics, rules or the model.
3. Switch to **Operations Manager** and ask about revenue. It is withheld, and the answer says so rather than pretending the metric does not exist.
4. Open <http://localhost:3002/admin/simulate>, run the **failure_burst** template, then go back and ask about transaction failure rate. The engine finds the change you just planted.

If something does not work

Symptom	Cause	Fix
Answers are generic or oddly short	Ollama is not reachable from Docker	<code>curl http://localhost:11434/api/tags</code> . If it fails, start Ollama and rerun step 1.
Answers quote a truncated set of facts	The narrator was built without <code>num_ctx 8192</code>	Redo the <code>ollama create</code> in step 1.
Dashboard shows zeros everywhere	Data is not loaded	Rerun step 4.
"No investigation has produced an insight yet"	Nothing has been scored	Rerun the three <code>rescore</code> calls in step 4.
A page will not load	A service is still starting	<code>docker compose ps</code> , then <code>docker compose logs <service></code> .

To stop everything: `docker compose down` . To also delete the warehouse: `docker compose down -v` .

NexaBank is a working retail bank — customers, accounts, transactions, loans and their KYC steps, cards, campaigns, branches. It generates real banking activity and the behavioural telemetry that accompanies it, and it holds the Simulate console that plants anomalies *in the source data*.

FinInsights watches that activity, decides whether a movement is trustworthy, finds the segment responsible, projects where it is heading, recommends an action with a named owner, and writes the finding in plain English — with every figure traceable to a stored piece of evidence.

The governing document is [CLAUDE.md](#). Read it before writing code. This README covers only how to run the thing.

Documentation

Markdown lives in [docs/](#) and nowhere else.

Document	What it answers
docs/SOLUTION.md	What we are building and why, in plain English
docs/ARCHITECTURE.md	How the pieces fit and how data flows between them
docs/DATA_MODEL.md	Bronze, Silver and Gold — what lives in each layer
docs/INTELLIGENCE.md	The agent, its six tools, and the verifier

Running it

Everything runs in Docker. Do not use a host `.venv`, a host `node`, or `npm run` — those environments drift from the images without warning.

```
docker compose up -d
```

Service	Port	What it is
dashboard	3001	FinInsights dashboard (Next.js)
nexabank-frontend	3002	NexaBank retail banking app (Next.js)
nexabank-backend	5000	NexaBank API + Simulate engine (Express/Prisma/Postgres)
ingestion-api	8000	Event intake, masking, deterministic event ids (FastAPI)
analytics-api	8001	Metric API + the intelligence layer (FastAPI)
pipeline	8003	Bronze/Silver/Gold transforms and the extract scheduler (FastAPI)
Ollama	11434	The local narrator model — runs on your machine, not in Docker
clickhouse	8123	The warehouse
broker	9092	Kafka — transport only, never the system of record

Every number and every finding is produced without the model. Setting `INTELLIGENCE_LLM=0` turns the narrator off and the engine still works end to end; only the prose changes.

Rebuilding after an edit

The three Python services (`ingestion-api` , `analytics-api` , `processor-worker`) bind-mount nothing. Their source is baked in at build time, so `--reload` is watching files that never change:

```
docker compose up -d --build analytics-api
```

The Node services (`nexabank-backend` , `nexabank-frontend` , `dashboard`) do bind-mount `src` , but neither watcher reliably sees a write through a Windows bind mount. Restart before judging a change:

```
docker compose restart nexabank-backend
```

Checking your work

```
# Query the warehouse
docker compose exec clickhouse clickhouse-client --password clickhouse \
  --query "SELECT kpi_id, count() FROM gold.kpi_daily GROUP BY kpi_id"

# Type-check a TypeScript project without touching host node_modules
docker compose exec dashboard npx tsc --noEmit

# Score the engine against the ground truth the seeder planted
docker compose --profile tools run --rm tools python scripts/run_intelligence_gates.py

# Confirm Kafka is carrying events rather than silently running the ClickHouse fallback
curl -s localhost:8000/health          # ingest_path: kafka | clickhouse_fallback
```

Rebuild status

The repository is mid-migration from a prototype into the structure CLAUDE.md section 12 describes. The state before the migration is recoverable at tag `pre-cleanup-2026-08-31` .

Phase	What it does	State
P0	Delete stale docs, junk, dead code	done
P1	Move to the section 12 folder structure; split the 4,176-line <code>api/main.py</code>	next
P2	Freeze the four interfaces; rewrite <code>contracts/</code> to the five-KPI chain	
P3	NexaBank generates the full banking domain; Simulate console with anomaly templates	
P4	Bronze / Silver / Gold in ClickHouse	
P5	Rebuild the intelligence agent against the live Metric API	

Secrets

Never commit one. `.env.example` lists every variable the stack reads; copy it to `.env` and fill it in. Credentials are read from the environment, never from source.